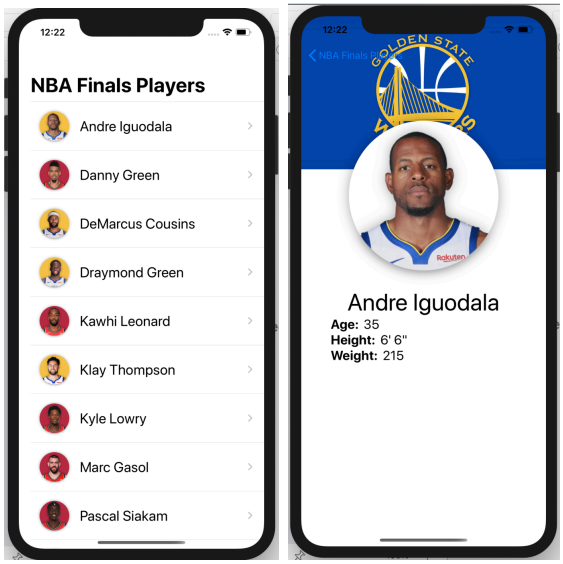
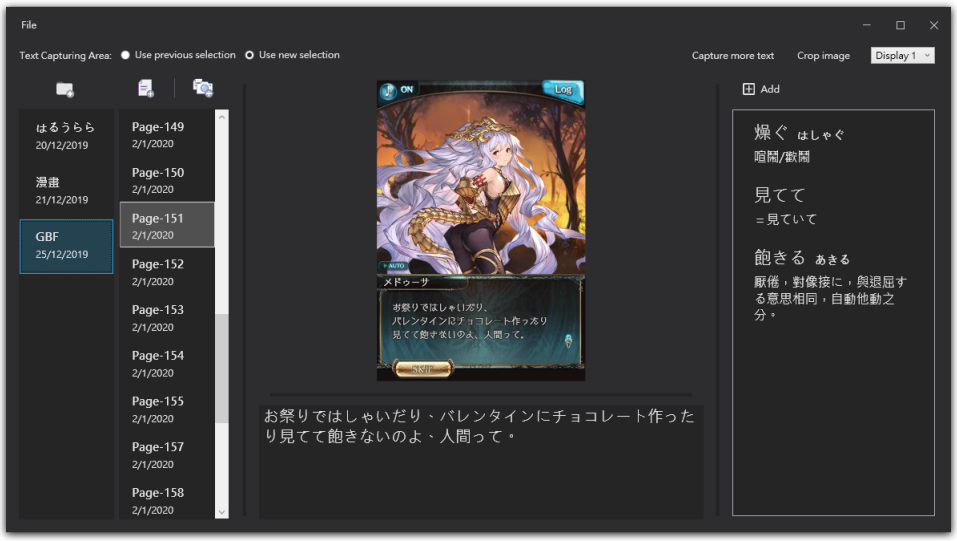
Projects

On Mobile Development. Having created my project II below on learning languages, I decided to learn how to build mobile application in order to view and save my screenshots, dictionaries, etc, on mobile. I have learnt how to do simple data fetching, how to do a navigation list using swift-UI and react-native. Below is one of them following a tutorial on swiftUI.



Personal Project II: Dictionary via Screencap. Source code has been uploaded to my github. I have been a hard-core player in two games: the granblue fantasy and the final fantasy 14. Both of them have created a gigantic world with a very rich story, but unfortunately the stories are not availalable in Chinese.

This serves as a motivation for me to create an application that: Without interrupting the gameflow, users can save each screenshot, can go back and review the content of each page, jot down their dictionary, and categorize their screenshots by "note":



Users can save their dictionaries for review. And they can capture the text from screenshot by the tool I developed in the previous project.

The application is created by WPF in .net framework and recently has been transited into .net-core 3.

Personal Project I: Capture Texts in Image. This is my first windows application in an attempt to assist my work in the current company.

I have been working on transforming old flash games into HTML5 games. Since it is impossible to copy down some animated texts from original .fla file, to extract the texts I make the following application.

This application was first developed in Python with a library called tkinter, and turned into a standalone .exe file by using pyinstaller (detail is also listed in my github). The journey of bug fixing prompt me to develop this application directly in C#.

Bootcamp Project: Stand Overflow (SO). Cowork with two other teammates. This Project cannot be demonstrated any more as the aws instance was shut down by my teammate.

Features. SO provides a platform for "background checking" regarding to the stand of a company towards controversial issues in Hong Kong. The nature of issues can be diversified, they can be political, environmental and educational.

Structure of This Application.

In frontend, we use

- React
- Redux

while in backend we use:

- Node.js + Express Framework
- PostgreSQL
- Elasticsearch
- Ethereum Blockchain Network

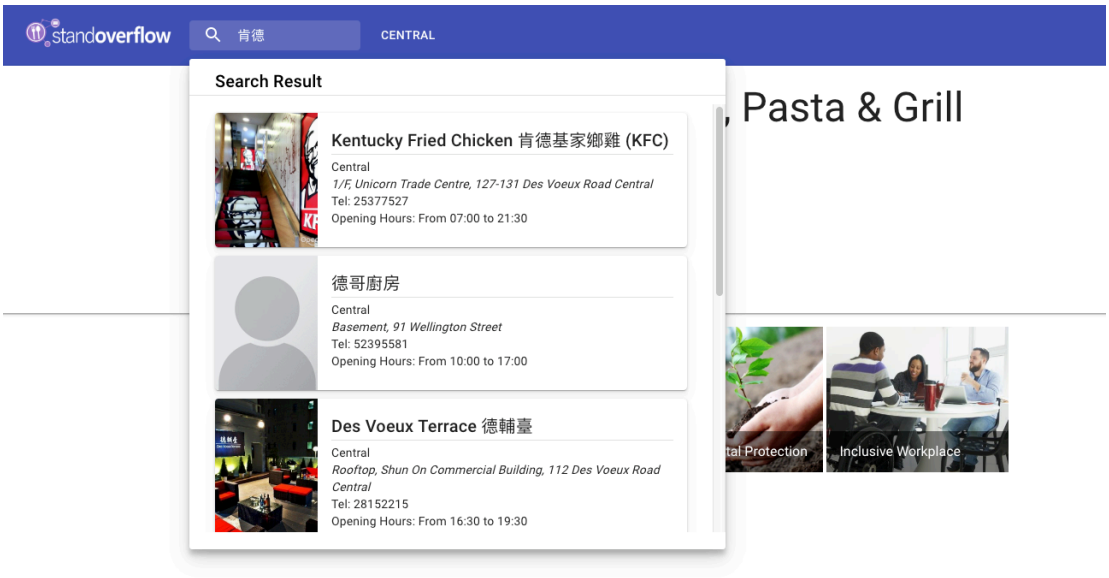


Figure 1: Search Function Inside our Application

As shown in Figure 1, users can search for the food store for which they want to have a review. Users can upload a review of a company on a specific issue as in the picture on the right as in Figure 2.

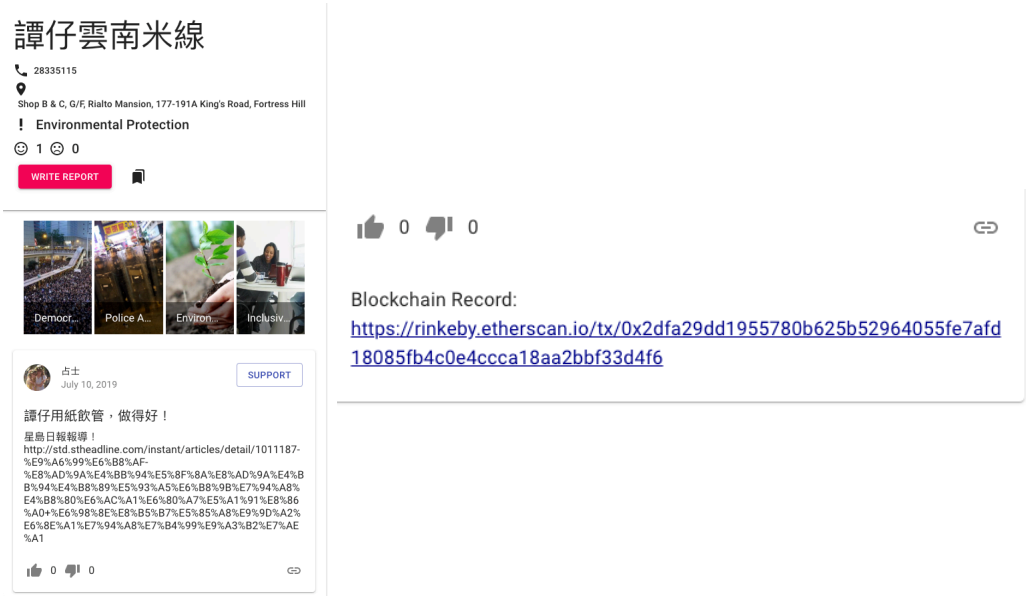


Figure 2: Review in Mobile

Each review is also sent to an Ethereum network and identified with an "id" called transaction hash (as shown in Figure 2 as well), by doing so no one can modify the content any more.

Implementation Detail of Blockchain Framework.

How it Works. For the sake of better user experience, we will let users save their record to the blockchain through our account.

Figure 3 is a smart contract written in solidity language. They are tested and deployed to the

blockchain network through Remix - Ethereum IDE. In SO we transfer data (instead of money) to this contract for record.

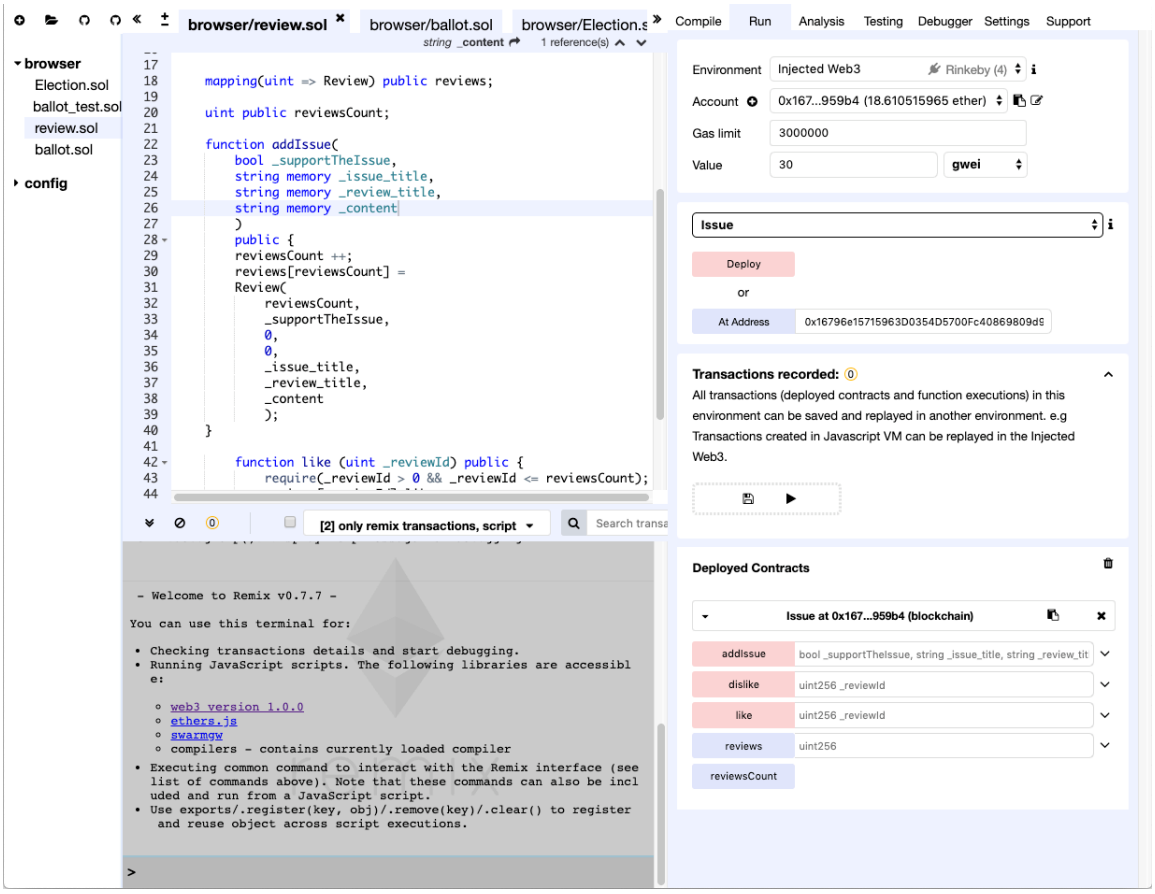


Figure 3: Writing a Solidity Contract

After a contract is deployed, we can connect to our Ethereum network account in the backend by using our private key as in Figure 4.

```
constructor(private knex: Knex, private web3: Web3) {
  this.knex;
  this.privateKey =
    "C62D3D048C3F[redacted]";

  this.account = this.web3.eth.accounts.privateKeyToAccount(
    "0x" + this.privateKey
  );
  this.web3.eth.accounts.wallet.add(this.account);
  this.web3.eth.defaultAccount = this.account.address;
  this.contractAddress = "0xc775d626004936c7D209657d6e602Fd892Ea2877";
  this.reviewsContract = new this.web3.eth.Contract(…
    this.contractAddress
  );
}

addReview = async (
  supportTheIssue: boolean,
  issueTitle: string,
  reviewTitle: string,
  content: string
): Promise<string> =>
  new Promise((resolve, reject) => {
    this.reviewsContract.methods
      .addIssue(supportTheIssue, issueTitle, reviewTitle, content)
      .send(
        {
          from: this.web3.defaultAccount,
          gasLimit: this.web3.utils.toHex(160000),
          gasPrice: this.web3.utils.toHex(this.web3.utils.toWei("30", "gwei"))
        },
        (err: string, hash: string) => {
          resolve(hash);
        }
      );
  });
```

Figure 4: Implementation Detail in Backend

Then users of SO can use our API to record their review to the blockchain network, and that's all we have done for the blockchain part.